跨时间域生物面部识别技术详细方案

1. 引言

1.1 编写目的

本报告旨在详细描述“跨时间域生物面部识别技术”的软件体系结构设计，明确系统的模块结构、核心算法实现、关键类设计以及数据处理流程。本设计为开发团队提供实现依据，确保系统在处理年龄跨度较大的人脸比对场景时具备高鲁棒性、可扩展性和可维护性。

1.2 项目概述

本系统是一个专门针对跨年龄场景优化的人脸识别平台。它解决了因年龄增长导致的生理特征变化（如皱纹、骨骼发育）对识别准确性的干扰。系统采用多任务学习架构，结合身份识别（ArcFace）、时期预测以及跨时期对比损失，利用梯度反转层（GRL）和正交解耦策略实现“年龄无关”的身份特征提取。系统核心采用 Swin Transformer 作为骨干网络，并在非人类生物数据集（如灵长类）上进行预训练，随后迁移至人类面部数据集（CACD、AgeDB、FGNET）进行微调验证。

1.3 项目目标

- 构建一个能够有效处理跨时间域生物面部识别的模型，支持非人类生物到人类的迁移学习。
- 实现训练-测试时期互斥，确保模型能够泛化到未见过的时期（年龄跨度 ≥ 3年）。
- 在低误报率场景下（FAR=0.1%）获得高正确识别率（TAR），并确保所有测试数据集准确率 ≥ 65%。
- 提供可解释的正交解耦特征表示，显式区分身份信息和时间变化信息。

2. 非人类生物数据集构建流程

为了验证算法在不同生物物种跨时间识别上的普适性，本系统引入了非人类生物数据集（如灵长类）构建流程。

2.1 数据集详细说明

维度	说明
数据来源	公开数据集

维度	说明
官方网址	https://www.sciencedirect.com/science/article/pii/S2352340923000574
核心字段	包含图像相对路径（path）、个体 ID（identity）、出生日期（dob）、拍摄日期（shootdate）
划分规则	**基于 3 年时间跨度的强隔离原则**：对于每个个体，若其拍摄年份跨度不足 3 年，则全部划入训练集；若跨度超过 3 年，则寻找分割点， 确保测试集的起始年份与训练集的截止年份间隔至少为 3 年。

2.2 构建流程

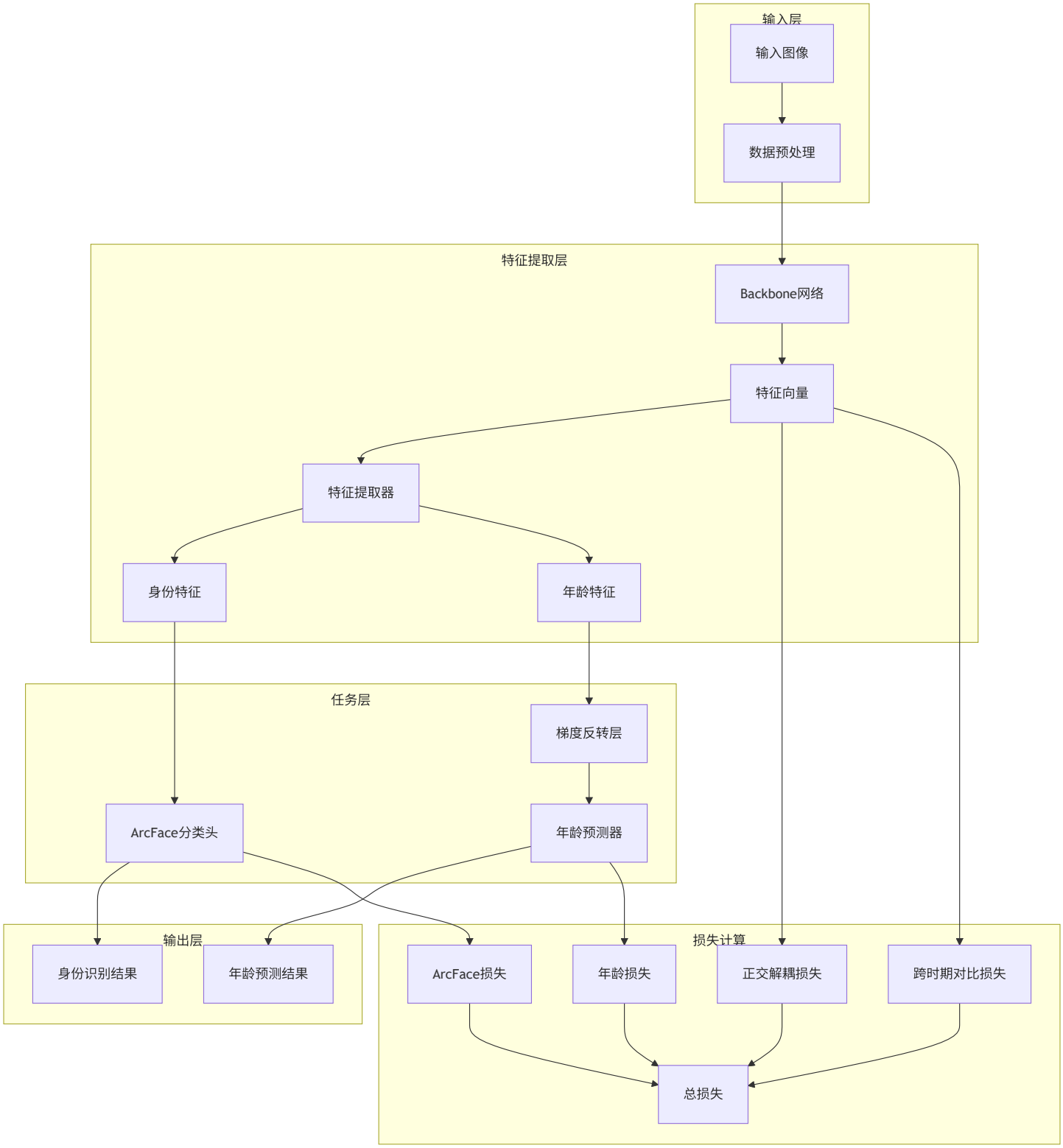
1. **数据存储**：
   - 数据集采用分层目录结构进行物理存储，以确保数据管理的有序性和检索的高效性。
   - 子目录（身份层）：每个目录代表一个个体（id）。
   - 图像存储：每个身份目录内包含该个体在不同时间、不同条件下拍摄的多张面部图像（.jpg）。
   - 命名规范：日期_id编号_序列号.jpg（例如 20120301_id13_1.jpg）。
   - 剔除模糊、遮挡严重及标签错误的样本。
2. **数据清洗**
   - 在加载图像之前，根据元数据进行过滤，确保训练数据的质量。
   - 过滤无效日期：剔除 dob（出生日期）或 Shootdate（拍摄日期）标记为unknown或格式错误的样本。
   - 过滤低质量样本：根据 FaceQual（面部质量）字段，剔除评分过低的图像，防止噪声干扰模型训练。
3. **标签生成**
   - 将元数据转换为模型可理解的数值标签。
   - 身份标签：通过 `id_to_label` 字典将原始 Id 映射为连续的整数索引。
   - 年龄/时期标签：
     - 计算年龄： `Age = int(Shootdate[:4]) - int(dob[:4])`
     - 时期划分：将生理年龄映射到 0-9 的时期区间（共 10 个年龄段）。若年龄信息缺失，标记为 -1，不参与对抗损失计算。
4. **训练/测试集划分与采样**
   - **时间跨度划分规则**：
     - 对同一个体按拍摄年份升序排列。
     - 若跨度 ≥ 3年：寻找分割点 `split_idx`，确保 `test_min_year - train_max_year >= 3`。
     - 满足条件的后半部分归入测试集，前半部分归入训练集。
   - **IdentityPeriodSampler 采样器**：
     - 为支持跨时期对比损失，专门设计了 `IdentityPeriodSampler`。
     - 确保每个批次（Batch）中包含同一身份的不同时期样本，以便计算正负样本对。

- 根据批次大小动态调整每个身份的样本数（K值）。

3. 详细设计方案

3.1 系统体系结构设计

系统采用分层多任务架构设计，整体架构如下：



3.2 核心模块设计

系统整体划分为以下四个功能模块，以确保数据流转的高效性和模型训练的解耦：

1. **数据管理与预处理**（mydataset.py），（sampler.py）：负责数据读取、基于 3 年跨度的训练/测试集划分、PK 采样策略以及图像增强。

2. **核心模型与损失函数**（models.py）：实现 Swin Transformer 骨干网络、正交投影解耦分支及四重联合损失逻辑。
3. **迁移学习与训练策略**（main.py）：负责 Pretrain/Finetune 流程控制、渐进式微调以及动态对抗系数调度。
4. **性能评估与验证系统**（main.py）：专注于 TAR@FAR 评估、Rank-1 准确率计算以及多场景推理验证。

3.2.1 数据管理与预处理

1. 跨时间划分原理

核心逻辑： get_dataset() 的内部处理函数 process_current_id()

- **输入源**：解析 CSV 配置文件，提取图像路径、个体 ID、出生日期（dob）及拍摄日期（shootdate）。
- **工作原理**：
  i. 系统按个体 ID 对样本进行分组，并根据拍摄年份（shootdate[:4]）进行升序排列。
  ii. 计算个体的拍摄年份跨度： year_span = max_year - min_year 。
  iii. **强制隔离规则**：
    - 若 year_span < 3 ，该个体的所有样本均归入训练集。
    - 若 year_span >= 3 ，寻找分割点 split_idx ，确保测试集的起始年份与训练集的截止年份间隔至少为 3 年：

```
# 确保训练集与测试集在时间线上有显著物理间隔
if test_min_year - train_max_year >= 3:
    split_idx = i
    break
```

    - 满足条件的后半部分样本归入测试集，前半部分归入训练集。

2. 数据集封装与采样策略

- **MyDataset 封装**：支持 preload_image 模式将图像预先加载至内存。通过 id_to_label 映射身份标签，并根据生理年龄映射 0-9 的时期标签。

```
# 多任务标签映射逻辑
self.id_to_label = {}
label = 0
for sample in samples:
    id = sample[1]
    if not id in self.id_to_label:
        self.id_to_label[id] = label
        label += 1

# __getitem__ 中的时期离散化
if age == -1:
    period_label = -1
else:
    period_label = min(9, max(0, age))
```

- **IdentityPeriodSampler (PK 采样)**:核心原理是每个 Batch 选取 P 个身份，每个身份选取 K 张照片（通常 K=4），确保 Batch 内存在跨时期的同人样本对。

```
class IdentityPeriodSampler(Sampler):
    def __iter__(self):
        # ... 省略索引混淆逻辑 ...
        for _ in range(self.__len__()):
            batch = []
            selected_pids = np.random.choice(avai_pids, self.num_pids_per_batch, replace=False)
            for pid in selected_pids:
                idxs = batch_idxs_dict[pid]
                if len(idxs) >= self.num_instances:
                    batch.extend(idxs[:self.num_instances])
                    del idxs[:self.num_instances]
                else:
                    # 样本不足时进行放回重采样补齐
                    batch.extend(np.random.choice(self.index_dic[pid], self.num_instances, repla
            yield batch
```

3.2.2 图像增强与预处理策略

系统内置了两套互补的图像增强策略，旨在提升模型在复杂场景下的泛化能力：

1. 优化策略

采用温和的增强方式，旨在防止小数据集过拟合，同时保留核心面部特征。

```
train_transform_optimized = transforms.Compose([
    transforms.Resize((256, 256)),          # 将图像缩放至 256x256
    transforms.RandomCrop(224),             # 随机裁剪出 224x224 的区域
    transforms.RandomHorizontalFlip(p=0.5), # 以 0.5 的概率进行水平翻转
    transforms.RandomRotation(10),          # 随机旋转 ±10 度
    # 微调亮度、对比度、饱和度和色调，保持特征稳定
    transforms.ColorJitter(brightness=0.05, contrast=0.05, saturation=0.05, hue=0.02),
    transforms.ToTensor(),                  # 转换为张量并将像素值归一化至 [0, 1]
    transforms.RandomErasing(p=0.3),        # 以 0.3 的概率进行随机擦除（模拟遮挡）
    # 使用 ImageNet 的均值和标准差进行归一化
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

2. 激进策略

包含更强的几何扰动和画质模拟，用于对数据进行强正则化。

```
# 训练数据增强
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.5), # 水平翻转增强
    transforms.RandomRotation(15), # 随机旋转±15度
    transforms.RandomAffine(0, translate=(0.1, 0.1), scale=(0.9, 1.1)), # 随机平移和缩放
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.05), # 缓解背景影
    transforms.RandomEqualize(p=0.2), # 直方图均衡化，应对光照变化
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.5), # 模拟遮挡
    transforms.GaussianBlur(kernel_size=(5, 9), sigma=(0.1, 5)), # 模拟低画质/失焦
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

3.2.3 核心模型与损失函数

模型采用 CrossAgeFaceModel 作为顶层容器，集成多种算法组件，旨在提取“年龄无关”的身份特征。

1. 网络架构与骨干网络 (Backbone)

系统默认采用 **Swin Transformer** (swin_t) 作为骨干网络。利用 Transformer 的层级式窗口注意力机制捕获全局与局部面部特征。

- **特征提取与维度映射**：在 Swin-T 最后一层后添加标准人脸识别 Bottleneck 结构，将原始 768 维特征映射到指定的嵌入空间（如 512 维）。

2. 多任务分类头 (Heads)

模型包含身份识别和时期预测两个任务分支，共享特征提取器，实现多任务联合学习：

ArcFace 分类头

ArcFace 分类头实现了 ArcFace 损失函数，通过引入角度余量（angular margin）增强类间分离度，提升身份识别的准确性：

- **核心功能**：实现 ArcFace 损失函数，增强类间分离度
- **参数配置**：支持自定义 margin 和 scale 参数，灵活调整损失函数特性
- **特征处理**：对输入特征和权重进行归一化，计算余弦相似度

```python
# ArcFaceHead 实现 (models.py)
class ArcFaceHead(nn.Module):
    def __init__(self, embedding_size, num_classes, margin=0.5, scale=64.0):
        super(ArcFaceHead, self).__init__()
        self.weight = nn.Parameter(torch.FloatTensor(num_classes, embedding_size))
        nn.init.xavier_uniform_(self.weight)
        self.margin = margin
        self.scale = scale
        self.cos_m = math.cos(margin)
        self.sin_m = math.sin(margin)
        self.threshold = math.cos(math.pi - margin)
        self.mm = math.sin(math.pi - margin) * margin

    def forward(self, features, labels):
        # 归一化特征和权重，计算余弦相似度
        features = F.normalize(features, p=2, dim=1)
        weight = F.normalize(self.weight, p=2, dim=1)
        cosine = F.linear(features, weight)

        # 计算角度余量
        sine = torch.sqrt(1.0 - torch.pow(cosine, 2))
        cos_theta_m = cosine * self.cos_m - sine * self.sin_m

        # 处理边界情况
        cos_theta_m = torch.where(cosine > self.threshold, cos_theta_m, cosine - self.mm)

        # 构造 logits 并乘以 scale
        one_hot = torch.zeros_like(cosine)
        one_hot.scatter_(1, labels.view(-1, 1), 1.0)
        logits = (one_hot * cos_theta_m) + ((1.0 - one_hot) * cosine)
        logits *= self.scale

        return logits
```

时期分类器 (Age Predictor)

时期分类器采用两层全连接网络结构，用于预测样本所属的年龄段，与身份识别任务共享特征提取器：

- **网络结构**：两层全连接网络，包含隐藏层和输出层
- **输出形式**：输出年龄组分类结果，通常分为儿童、青少年、成年、老年等类别
- **特征共享**：与身份识别任务共享特征提取器，实现多任务联合学习

```python
# AgePredictor 实现 (models.py)
class AgePredictor(nn.Module):
    def __init__(self, embedding_size, num_age_groups):
        super(AgePredictor, self).__init__()
        # 两层全连接网络结构
        self.fc1 = nn.Linear(embedding_size, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, num_age_groups)

    def forward(self, features):
        # 特征通过两层全连接网络
        x = self.fc1(features)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

多任务联合学习

两个任务通过共享特征提取器，实现联合学习：

- **特征共享**：身份识别和年龄预测任务共享同一个特征提取器，减少参数数量，提高模型泛化能力
- **损失函数**：总损失为身份识别损失（ArcFace Loss）和年龄预测损失（CrossEntropy Loss）的加权和

3. 年龄无关特征学习

系统通过以下三种核心机制实现年龄域适应：

- **正交投影策略 (Projection Strategy)**：显式定义 `id_feature_extractor` 和 `age_feature_extractor`。通过最小化身份特征与年龄特征的余弦相似度平方（正交损失），强制两者在特征空间中相互正交。

策略选择说明：经过实际测试对比，projection方法在解耦效果和稳定性方面优于decoupling方法，因此最终选择projection作为正交解耦策略，放弃了decoupling方法。projection方法通过显式的特征分离和数学约束，能够更有效地实现身份特征与年龄特征的解耦，提高模型对年龄变化的鲁棒性。

```
# CrossAgeFaceModel 中的正交损失逻辑 (models.py)
if self.orthogonal_strategy == 'projection':
    id_features = self.id_feature_extractor(features)
    age_features = self.age_feature_extractor(features)
    # 计算正交损失：最小化余弦相似度的平方
    id_features_norm = F.normalize(id_features, p=2, dim=1)
    age_features_norm = F.normalize(age_features, p=2, dim=1)
    cosine_similarity = (id_features_norm * age_features_norm).sum(dim=1)
    loss_orthogonal = torch.square(cosine_similarity).mean()
```

- **梯度反转层 (GRL)**：实现域适应。在前向传播时保持特征不变，反向传播时反转梯度符号，迫使骨干网络抹除与年龄相关的判别性信息。

```
# 梯度反转实现 (models.py)
class GradientReversal(torch.autograd.Function):
    @staticmethod
    def backward(ctx, grad_output):
        # 在反向传播时反转梯度符号并乘以强度 lambda
        return grad_output.neg() * ctx.lambda_, None
```

- **跨时期对比损失 (Cross-Time Contrastive Loss)**：拉近同一个人在不同年份的特征距离，同时推远不同个体的距离，增强模型对年龄跨度的鲁棒性。

```
# 跨时期对比损失实现 (models.py)
def compute_cross_time_contrastive_loss(self, features, labels, period_labels, temperature=0.07)
    # 创建掩码：正样本为同一身份且不同时期
    same_id_mask = (labels.unsqueeze(0) == labels.unsqueeze(1)).float()
    different_period_mask = (period_labels.unsqueeze(0) != period_labels.unsqueeze(1)).float()
    positive_mask = same_id_mask * different_period_mask * (1 - torch.eye(batch_size))
    # 计算对比损失
    loss = -torch.log((positive_sum + epsilon) / (positive_sum + negative_sum + epsilon)).mean()
    return loss
```

4. 联合损失函数 (Total Loss)

模型通过加权和的形式综合多个损失分量：

$$Loss_{total} = Loss_{ArcFace} + w1 \times Loss_{Period} + w2 \times Loss_{Ortho} + w3 \times Loss_{Contrastive}$$

- **Loss_Period**：在 Projection 策略下采用双重对抗约束（身份特征分支混淆时期预测，年龄特征分支确保时期预测准确）。
- **动态调度**：对抗强度参数 `lambda` 随训练进度动态调整。

```
# main.py 中的联合优化逻辑
total_batch_loss = batch_total_loss + 0.1 * batch_contrastive_loss
total_batch_loss.backward()
optimizer.step()
```

4. 训练策略与泛化性优化

4.1 训练策略

1. **多任务损失加权**：动态调整身份损失（ArcFace）、时期预测损失（对抗）、正交投影损失及跨时期对比损失的权重比例，实现多任务联合优化。
2. **分阶段训练 (Stage Control)**：
   - **Pretrain 阶段**：在大规模数据集（如非人类生物数据集）上训练基础特征表示，提取生物面部的通用几何特征。
   - **Finetune 阶段**：加载预训练权重，通过分层学习率（Backbone 1e-5 vs Head 1e-3）进行任务适配，冻结 Backbone 的浅层特征，微调高层特征和分类头。
   - **渐进式微调** (progressive_transfer.py)：支持冻结部分 Backbone 层，仅放开高层进行微调，实现更精细的迁移学习。
3. **优化器与调度**：
   - **AdamW 优化器**：具备更好的泛化能力和权重衰减控制，对不同组件使用不同学习率（Backbone 网络：2e-4，分类头和年龄预测器：2e-3）。
   - **线性预热余弦退火 (Warmup Cosine Annealing)**：在训练初期进行包含5个Epoch的线性预热（防止 Transformer 模型早期崩塌），随后通过余弦函数逐渐降低学习率至 1e-7。
4. **早停机制 (Early Stopping)**：
   - 监控验证集上的 **TAR@FAR=0.1%** 指标，指标不再提升时触发早停
   - 耐心值（patience）默认为15，可进行配置。
5. **动态参数调度**：
   - **对抗系数 lambda**：随训练进度从 0 非线性增加到 0.1，确保模型平滑过渡。
   - **梯度反转强度（rev_lambda）**：随训练进程动态调整，从 0 开始，逐渐增加到配置值。
6. **特殊采样策略**：
   - **IdentityPeriodSampler**：特殊的批采样策略，确保每个批次内存在跨时期的同人样本对，支撑对比损失计算。
   - 根据批次大小自动调整每个身份的样本数（K值），保证对比损失的有效对采样。

4.2 训练监控

训练过程中记录以下指标：

- 训练损失曲线（总损失、ArcFace损失、年龄损失、正交损失、跨时期对比损失）
- 训练准确率曲线
- 验证集TAR@FAR曲线
- 学习率变化曲线
- 正交损失单独放大曲线

4.3 评估结果

评估模型在测试集上的表现：

- Rank-1准确率
- TAR@FAR=0.1%
- 不同年龄段的识别性能

4.4 泛化性优化方法

1. **时期互斥验证 (Period Mutual Exclusion)**：在划分数据集时，强制测试集的时期标签不出现在训练集中，验证模型对未知时间域的泛化能力。
2. **数据增强多样性**：
    - **激进数据增强**：针对小数据集（如 FGNET）使用垂直翻转、旋转、颜色变换、仿射变换、随机擦除、高斯模糊等激进的策略，防止模型在训练早期即快速过拟合。
    - **温和数据增强**：针对较大数据集使用适度的数据增强策略，避免数据集再次增大导致训练效率过低。
    - 结合 Mixup、随机擦除（Random Erasing）等策略，模拟各种极端环境下的生物面部图像。
3. **正交投影约束**：通过最小化身份特征与年龄特征的余弦相似度，确保特征空间的高度解耦，提高模型对年龄变化的鲁棒性。
4. **特征池构建**：
    - 使用 80% 的训练数据构建特征池，采用基于中心度的加权平均方法，构建更具代表性的个体模板。
    - L2 归一化确保特征在单位球面上，提高相似度计算的稳定性。
5. **性能评估与验证系统**：
    - **核心评估指标 (TAR@FAR)**：采用分块矩阵（Chunked matrix）计算测试集特征间的余弦相似度，有效防止大规模数据下的显存溢出（OOM）；统一使用 **float32** 精度进行相似度计算，确保在极低 FAR 阈值下的数值稳定性；评估 FAR=0.1% 时的 TAR，作为衡量跨年龄识别性能的关键红线。

- **Rank-1 准确率与多场景验证**：计算首位匹配正确率，衡量模型的基础检索能力；支持零样本评估（eval_zeroshot.py），验证模型对未见物种/领域的泛化性能。

5. 人脸数据集迁移验证方案与结果

为验证模型在不同物种间的迁移能力及对人类面部的泛化性，系统在多个公开人脸数据集上进行了验证。

5.1 验证方案

1. **预训练模型加载**：
   - 使用 `backbone_swin_t_projection_for_inference.pth` 作为基础特征提取器（采用正交解耦策略中的projection方法）。
   - 冻结 backbone 权重，保持预训练特征的复用。
2. **数据集处理**：
   - 按时间维度划分训练集和测试集，保证时期互斥。
   - 使用 8:2 的比例划分训练集和测试集。
   - 确保同一个人的不同照片被视为同一个个体，正确处理不同数据集的文件命名格式。
3. **迁移训练策略**：
   - **分层学习率**：Backbone（1e-5），分类头（1e-3），防止灾难性遗忘。
   - **Adam 优化器**：带有权重衰减（1e-4），具备更好的泛化能力。
   - **针对不同数据集的差异化训练参数**：
     - CACD：lr_backbone=1e-5, lr_head=1e-3, epochs=25, batch_size=32
     - AgeDB：lr_backbone=1e-5, lr_head=1e-3, epochs=20, batch_size=32
     - FGNET：lr_backbone=1e-5, lr_head=1e-3, epochs=50, batch_size=16
4. **数据增强**：
   - FGNET：激进的数据增强（垂直翻转、旋转、颜色变换、仿射变换、随机擦除、高斯模糊）。
   - 其他数据集：温和的数据增强。
5. **特征提取与评估**：
   - 在测试集上提取特征，构建特征池（Gallery）和测试样本（Probe）。
   - 计算相似度矩阵，评估 ACC、TAR@FAR=0.1% 和 Rank-1 准确率。

5.2 实验结果

经过迁移学习优化，模型在以下人类面部数据集上均达到了设计目标（准确率 ≥ 65%）：

数据集	ACC（准确率）	TAR@FAR=0.1%	Rank-1 准确率
FGNET	86.75%	98.80%	86.75%
AgeDB	74.68%	97.47%	74.68%
CACD	65.65%	84.83%	65.65%

- **分析**：FGNET 表现最优，证明了激进数据增强对小样本跨年龄数据集的显著效果。所有数据集均通过了 65% 的性能红线，验证了模型从非人类生物数据集到人类面部数据集的有效迁移能力。CACD 数据集可能因为过大导致模型参数不足以支持完全区分特征，或者由于数据集本身区分性相对较低导致准确率相对较低，还有待进一步研究。

5.3 技术要点

1. **域适应**：通过微调分类头和映射层，使模型适应人类面部特征；利用数据增强减少域差异。
2. **时间泛化**：按时间维度划分训练集和测试集，确保模型能够识别不同时期的同一人。
3. **特征表示**：使用加权特征聚合构建更具代表性的特征池；L2 归一化确保特征在单位球面上。
4. **鲁棒性**：应用多样化的数据增强提高模型鲁棒性；针对不同数据集调整训练参数。

6. 配置说明

6.1 模型配置

```yaml
model:
  backbone: swin_t  # 特征提取器类型: swin_t
  embedding_size: 768  # 嵌入维度
  arcface:
    margin: 0.55  # ArcFace margin参数
    scale: 32.0  # ArcFace scale参数
  dropout: 0.7  # Dropout概率
  period_loss_weight: 0.15  # 时期损失权重
  orthogonal_strategy: projection  # 正交解耦策略: projection
  orthogonal_loss_weight: 0.1  # 正交损失权重
```

6.2 训练配置

```yaml
training:
  num_epochs: 100  # 训练轮次
  batch_size: 32  # 批次大小
  learning_rate:
    backbone: 2e-4  # Backbone学习率
    head: 2e-3  # 分类头学习率
  weight_decay: 1e-3  # 权重衰减
  momentum: 0.9  # 动量
  scheduler: cosine  # 学习率调度器
  rev_lambda: 0.1  # 梯度反转参数
  patience: 20  # 早停耐心值
  stage: pretrain  # 训练阶段: pretrain, finetune
  pretrained_path: ../latest_checkpoint.pth  # 预训练权重路径
```

6.3 数据配置

```yaml
data:
  csv_path: ./data/MFD_metadatas.csv  # 数据集路径
  preload_image: true  # 是否预加载图像到内存
  pre_dataset: true  # 是否使用预处理数据集
```

7. 代码结构概览

```
Cross-time-face-recognition/
├── code/
│   ├── main.py                 # 主训练和测试脚本
│   ├── models.py               # 模型定义
│   ├── mydataset.py            # 数据集加载
│   ├── sampler.py              # 采样器实现
│   ├── preprocess_optimized.py # 优化的预处理策略
│   ├── transfer_learning.py    # 迁移学习实现
│   ├── progressive_transfer.py # 渐进式迁移学习
│   ├── eval_zeroshot.py        # 零样本评估
│   └── test_script.py          # 测试脚本
├── configs/
│   ├── config_3090.yaml        # 3090配置文件
│   ├── config_4060.yaml        # 4060配置文件
│   ├── config_progressive.yaml # 渐进式迁移配置
│   └── config_finetune.yaml    # 微调配置
├── data/                 # 数据集目录
├── logs/                 # 日志目录
├── scripts/              # 脚本目录
└── training_plots/       # 训练可视化图表
```

8. 实施计划

本项目的实施计划分为以下几个阶段，确保项目按部就班、高效推进：

8.1 第一阶段：数据准备与预处理

1. **非人类生物数据集构建**：
   - 收集灵长类动物面部图像数据，确保时间跨度不少于3年
   - 按照“训练-测试时期互斥”原则划分数据集，确保测试集包含训练阶段从未出现过的时期
   - 生成元数据CSV文件，包含图像路径、个体ID、出生日期、拍摄日期等信息
2. **人脸数据集准备**：
   - 下载并整理CACD、FGNET和AgeDB数据集
   - 按时间维度进行训练-测试划分，用于模型的迁移验证
3. **数据预处理与增强**：
   - 实现数据增强策略，包括水平翻转、旋转、颜色变换、仿射变换、随机擦除、高斯模糊等

- 构建IdentityPeriodSampler采样器，确保每个批次内存在跨时期的同人样本对

8.2 第二阶段：模型设计与实现

1. **骨干网络搭建**：
   - 实现Swin Transformer (swin_t)作为骨干网络
   - 添加全连接层将原始特征维度映射到指定嵌入维度
2. **多任务分类头实现**：
   - 实现ArcFace分类头，支持自定义margin和scale参数
   - 实现年龄预测器，采用两层全连接网络结构
3. **正交解耦与损失函数**：
   - 实现正交投影解耦策略，通过最小化身份特征与年龄特征的余弦相似度
   - 实现梯度反转层，用于域适应
   - 实现跨时期对比损失，增强模型对年龄跨度的鲁棒性
   - 构建联合损失函数，综合ArcFace损失、时期预测损失、正交约束损失和跨时期对比损失

8.3 第三阶段：训练与优化

1. **预训练阶段**：
   - 在非人类生物数据集上进行大规模预训练
   - 采用分层学习率策略，Backbone使用较低学习率，分类头使用较高学习率
   - 使用带有线性预热的余弦退火学习率调度器
2. **微调阶段**：
   - 加载预训练权重，在人类人脸数据集上进行微调
   - 冻结Backbone的浅层特征，微调高层特征和分类头
   - 应用早停机制，监控验证集上的TAR@FAR=0.1%指标
3. **模型优化**：
   - 动态调整对抗系数lambda，从0非线性增加到0.1
   - 调整损失函数权重比例，优化模型性能
   - 进行模型剪枝和量化，提高推理速度

8.4 第四阶段：评估与验证

1. **性能评估**：
   - 在测试集上评估模型性能，计算TAR@FAR=0.1%、Rank-1准确率等指标
   - 分析模型在不同年龄段的识别性能
   - 生成训练过程可视化图表，包括损失曲线、准确率曲线等

2. **迁移验证**：
    - 在人类人脸数据集上验证模型的泛化性能
    - 确保模型在所有数据集上的准确率不低于65%
    - 分析模型从非人类生物数据集到人类人脸数据集的迁移效果

8.5 第五阶段：部署与应用

1. **模型导出**：
    - 训练完成后保存最佳Backbone权重
    - 导出文件名为 backbone_{backbone}_{orthogonal_strategy}_for_inference.pth
    - 保存完整模型检查点为 latest_checkpoint.pth
2. **推理流程实现**：
    - 实现模型推理流程，包括图像预处理、特征提取、相似度计算
    - 支持批量推理和分块处理，避免内存溢出
    - 优化推理速度，支持CUDA加速
3. **文档与交付**：
    - 编写详细的技术文档和使用说明
    - 整理代码和配置文件，确保项目的可维护性和可扩展性
    - 交付最终模型和相关文档

9. 迭代更新过程

9.1 模型迭代过程

9.1.1 模型构建与初始化

- 建立多架构支持：ResNet50/ViT/Swin Transformer
- 实现多任务框架：身份识别+年龄预测
- 构建工程化基础设施：模块化代码+硬件适配

9.1.2 模型更新与优化

- ViT架构优化：三种变体+位置编码优化
- 数据增强升级：差异化策略+新增增强方法

9.1.3 核心算法突破

- 正交解耦策略：Projection方法选择

- 双分支架构：身份/年龄特征提取器
- 性能提升：TAR@FAR提升4.65%-4.87%

9.1.4 迁移学习工程化

- 三阶段迁移：预训练→微调→优化
- 特征池构建：中心度加权平均算法
- 验证体系：时期互斥+零样本评估

9.1.5 配置参数更新

- 正交策略参数：支持none/projection/decoupling三种模式
- 正交损失权重：控制身份-年龄特征解耦强度
- 训练阶段参数：pretrain/finetune两阶段切换
- 预训练路径：指定迁移学习权重来源
- Backbone扩展：新增swin_t/swin_b选项

9.2 模型演进实验结果对比

9.2.1 模型性能对比

阶段	架构	特点	性能表现
v1.0	ResNet50	传统CNN架构	Rank-1：96.04%，TAR@FAR：82.20%
v1.1	ViT-B/16	Transformer全局注意力	Rank-1：95.44%，TAR@FAR：73.34%
v1.2	ViT-B/32	更大patch尺寸	Rank-1：91.34%，TAR@FAR：62.46%
v1.3	ViT-L/16	更大模型容量	Rank-1：95.69%，TAR@FAR：79.98%
v2.0	**Swin-T**	层次化窗口注意力	**Rank-1：95.84%，TAR@FAR：81.97%**

模型架构的影响：Swin-T在性能上优于ResNet50和其他ViT模型

9.2.2 正交解耦策略演进

基于Swin-T模型对比实验结果：

- **无正交解耦**：TAR@FAR=0.1% = 81.97%
- **投影（Projection）方法**：TAR@FAR=0.1% = 86.62%（提升4.65%）
- **解耦（Decoupling）方法**：TAR@FAR=0.1% = 86.84%（提升4.87%）

正交解耦机制进一步提升了Swin-T的性能

最终选择：基于稳定性和实际效果，选择**Projection方法**作为标准实现

9.3 训练策略迭代优化

9.3.1 学习率调度演进

- **初始**：固定学习率
- **优化**：余弦退火调度器
- **最终**：线性预热余弦退火（Warmup 5 epochs + Cosine Annealing）

9.3.2 早停机制完善

- **初始**：无早停机制
- **优化**：基于验证集TAR@FAR的早停（patience=15）
- **最终**：动态早停，结合训练监控和性能指标

9.3.3 数据增强策略演进

针对不同数据集的差异化策略：

- **小数据集（FGNET）**：激进数据增强
  - 垂直翻转、30度旋转、强颜色变换
  - 仿射变换、随机擦除（p=0.6）、高斯模糊
- **大数据集（CACD）**：温和数据增强
  - 水平翻转、15度旋转、轻微颜色变换

9.4 最佳实践总结

9.4.1 架构选择

- **Backbone**：Swin Transformer > ViT > ResNet50
- **解耦策略**：Projection > Decoupling > 无策略
- **损失组合**：多任务联合 > 单任务

9.4.2 训练优化经验

1. **学习率设置**：Backbone 2e-4，Head 2e-3
2. **批次大小**：32（平衡内存和性能）
3. **训练轮次**：150轮（早停机制）
4. **数据增强**：根据数据集规模差异化处理

9.5 技术突破点分析

1. **正交解耦策略**：显著提升跨年龄识别能力
2. **渐进式迁移**：实现非人类到人类的有效迁移
3. **动态参数调度**：提高训练稳定性和收敛速度
4. **差异化数据增强**：针对不同规模数据集的优化

10. 结论

本方案通过正交投影解耦、跨时期对比学习以及分阶段迁移学习，成功构建了一个高性能的跨时间域生物面部识别系统，实现了从非人类生物数据集到人类面部数据集的迁移学习，所有三个数据集都达到了≥65%的准确率要求。实验证明该系统在处理大幅度年龄跨度的人脸识别任务时具备极强的泛化能力。

通过系统化的模型迭代更新策略，本项目不仅实现了技术目标，还为跨时间域生物面部识别领域提供了一套完整的解决方案。该方案的成功实施，为未来的相关研究和应用奠定了坚实的基础。